# Cahier des charges projet Csharp

## 1. Objectif du Projet

L'objectif de ce projet est de concevoir et de développer un jeu vidéo en langage C#. Ce projet sert d'évaluation pratique globale et a pour but de valider votre maîtrise des concepts de la Programmation Orientée Objet (POO) abordés en cours.

Vous êtes libres de choisir le jeu de votre choix du moment qu'il vous permet de remplir ce cahier des charges. Vous pouvez reprendre l'un des deux exemples de jeux que nous avons conçu en cours (RPG et City builder) mais il vous faudra grandement les modifier pour que ce soit véritablement votre jeu.

Vous devrez concevoir une architecture logicielle robuste, évolutive et organisée, en séparant la logique métier de l'affichage.

---

## 2. Contraintes Techniques et Architecturales

Votre projet doit obligatoirement intégrer les concepts de POO suivants. Une attention particulière sera portée à la pertinence de leur utilisation (ne les forcez pas, justifiez-les par le game design).

- **Architecture globale :** Le jeu doit faire intervenir au moins 20 classes différentes.
- **Héritage :** Ces classes doivent être organisées de façon logique et arborescente.
- **Abstraction :** Utilisation pertinente de classes abstraites et de méthodes abstraites.
- **Contrats :** Création et implémentation d'interfaces.

- **Polymorphisme :** Exploitation du polymorphisme (d'héritage, d'interface, ou idéalement des deux) pour le traitement générique des objets du jeu.
- **Gestion des erreurs :** Création et levée d'exceptions personnalisées.
- **Communication inter-objets :** Implémentation du Pattern Observateur et utilisation des Événements pour gérer les interactions (ex: mise à jour de l'UI quand un joueur perd des points de vie, système de succès, etc.).

---

## 3. Phases de Développement

Le projet est pensé pour être développé de manière itérative :

1. **Phase 1 - Version Console (Obligatoire) :** Commencez par concevoir le cœur du jeu contenant la logique et proposez une interface textuelle jouable directement dans le terminal. Cette étape permet de s'assurer que votre architecture POO est solide et fonctionnelle sans se laisser distraire par les graphismes.
2. **Phase 2 - Version Graphique via Unity (Bonus) :** Si vous êtes à l'aise avec la première phase et que votre logique métier est bien découplée, vous pouvez tenter d'adapter votre jeu avec une interface graphique (2D ou 3D) en utilisant le moteur Unity.

---

## 4. Livrables Attendus

Vous devrez fournir trois livrables distincts :

### A. Le Code Source

L'intégralité du code source du projet devra être hébergée sur un dépôt GitHub. L'historique des commits permettra d'évaluer la progression de votre travail et de votre équipe. Le dépôt doit contenir un fichier `README.md` expliquant comment lancer le jeu.

### B. Le Rapport Écrit

Ce document synthétique servira à justifier vos choix techniques et de conception. Il doit impérativement contenir les sections suivantes :

- **Présentation du jeu :** Concept, règles principales, et inspirations (jeux existants).
- **Simplifications :** Les concessions faites par rapport au concept initial pour respecter les délais.
- **Étapes de conception :** Déroulement chronologique de votre travail.

- **Architecture logicielle :** Explication détaillée de la structure de vos classes. *Note : L'intégration d'un diagramme de classes est fortement recommandée pour cette section.*
- **Interfaces :** Explication du rôle et de la pertinence des interfaces que vous avez conçues.
- **Travail futur :** Les pistes d'amélioration envisagées mais non implémentées faute de temps.

### C. La Soutenance Orale

Une présentation de votre travail devant la classe.

- **Durée :** Prévoyez une dizaine de minutes et 5 minutes de questions.
- **Contenu :** Présentation du concept, mise en avant des choix architecturaux majeurs, et démonstration jouable. L'utilisation d'un support visuel (même basique) est conseillée.

---

## 5. Barème d'Évaluation

La note finale prendra en compte votre architecture, la qualité de votre code et vos livrables.

Il y a 19 points à récupérer hors Unity et 6 points si vous proposez une version jouable en 2D ou 3D grâce à Unity. **Vous pouvez donc avoir plus de 20/20 vu qu'il y a 25 points au total**. Si votre note totale dépasse 20/20 les points excédentaires seront reportés et ajoutés à la note de votre examen écrit.

| Critère d'évaluation | Points | Détails |
|---|---|---|
| **Concepts de POO** | **/ 10 pts** | |
| *Architecture et volume* | 4 pts | Présence d'au moins 20 classes judicieusement organisées (héritage, abstraction). |
| *Contrats et comportements* | 2 pts | Présence et utilisation pertinente d'interfaces et de polymorphisme. |
| *Robustesse* | 2 pts | Création et gestion d'exceptions personnalisées métier. |
| *Design Patterns* | 2 pts | Implémentation d'événements et du pattern Observateur. |
| **Qualité du code** | **/ 2 pts** | Lisibilité, nommage (conventions C#), commentaires, encapsulation. |
| **Présentation orale** | **/ 2 pts** | Clarté du discours, pertinence des supports, qualité de la démo. |
| **Rapport écrit** | **/ 3 pts** | Structure, justification des choix, clarté du diagramme, recul critique. |

| Critère d'évaluation | Points | Détails |
| --- | --- | --- |
| **Interface de jeu** | **/ 2 pts** | Qualité et jouabilité de la version textuelle terminal. |
| **Bonus Unity** | **/ 6 pts** | La réalisation d'une interface graphique 2D ou 3D fonctionnelle via Unity. |
| **TOTAL** | **/ 25 pts** | |